Obiekt, odczyt i artefakt w 0!n

Kwalifikacja istniejących obiektów i tworzenie nowych struktur - od idei architektonicznej do mechanizmu w Pythonie

Praca problemowa i techniczna. Wydanie R1, 17 października 2026.

Streszczenie

Artefaktyzacja ustanawia rozliczalny interfejs obiektu w systemie 0!n. Łączy tożsamość, właściwości, powierzchnie współpracy, relacje i pochodzenie z jawnym stanem wiedzy. Ten sam szkielet może opisywać plik, funkcję, ideę, wynik badania, proces lub całą aplikację.

Praca zestawia dwie drogi: odczyt istniejącej reprezentacji oraz konstrukcję rozpoczynającą się od potrzeby. Wyjaśnia rolę czternastu powierzchni ośmiościanu ściętego, wersjonowanego profilu, czytnika, parametrów i kontraktów. Pokazuje, jak rekonstrukcja kodu i projektowanie spotykają się w sprawdzalnym modelu oraz w pętli kolejnych odczytów.

Część techniczna opiera się na przypiętym Studio R6 0.6.0 i przekazanym pakiecie wiedzy. Zawiera dwa przykłady Pythona, rozróżnia deklarację, obserwację, projekt i wykonanie oraz określa granice automatyzacji obecnej implementacji. Projektowany kierunek ogólnej fabryki rozwiązań pozostaje odróżniony od działającego rdzenia odczytu i modelowania.

Słowa kluczowe: artefakt, obiekt, profil odczytu, metaparametryzacja, relacja, rekonstrukcja, kompozycja, broker, ewolucja.

Spis treści

Wprowadzenie: jednostka opisu, z której można budować system 3

1. Obiekt istnieje w ramie pytań i odniesień 3

2. Kwalifikacja jest wielowymiarowym rozpoznaniem 4

3. Ośmiościan ścięty jako wspólna gramatyka interfejsu 5

4. Automatyczna artefaktyzacja: rzeczywisty algorytm R6 6

5. Przykład: odczyt wartości i świadomie zaprojektowany profil 8

6. Rekonstrukcja z kodu: co da się odzyskać automatycznie 9

7. Tworzenie artefaktu: od potrzeby do sprawdzalnego kandydata 10

8. Jak części tworzą spójny artefakt i jak artefakt tworzy proces 12

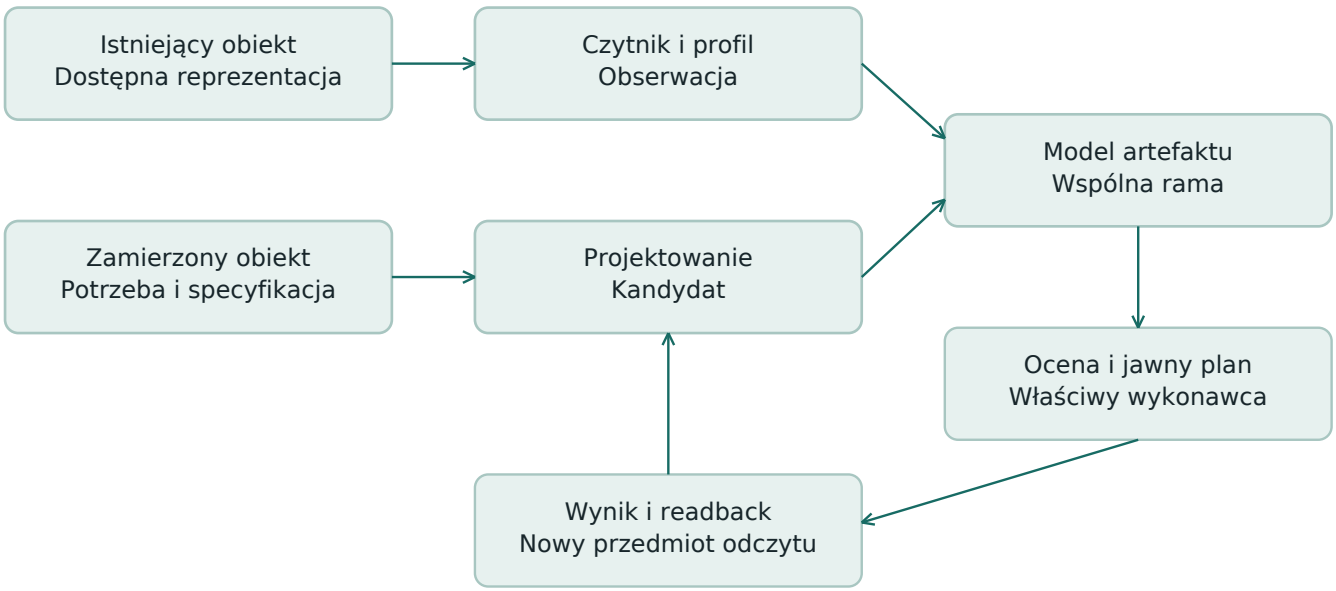
9. Broker, modele i próg materialnego działania 13

10. Stały odczyt, pamięć i pętla ewolucyjna 14

Zakończenie: wspólna forma dla poznawania i konstruowania 15

Źródła i podstawa weryfikacji 16

Dwie drogi do wspólnej reprezentacji



Schemat pojęciowy. Przejście od modelu do efektu wymaga oceny, jawnego zakresu i właściwego wykonawcy. R6 realizuje poszczególne opisane mechanizmy; diagram nie deklaruje jednego automatycznie uruchomionego obiegu całej federacji.

Wprowadzenie: jednostka opisu, z której można budować system

Z perspektywy 0!n dowolny obiekt jest przedmiotem, który można wyodrębnić w określonej przestrzeni założeń, zidentyfikować i poddać pytaniom o jego właściwości, granice oraz relacje. Takim przedmiotem może być plik, funkcja, katalog, kontrakt, wiadomość, wynik doświadczenia, hipoteza, proces, model, broker albo cała aplikacja. Artefakt jest sposobem włączenia tego przedmiotu do wspólnej architektury: nadaje mu adresowalną reprezentację, określa powierzchnie współpracy i zachowuje podstawę, na której system coś o nim twierdzi.

Proponuję następującą tezę pracy: **artefaktyzacja ustanawia rozliczalny interfejs obiektu w systemie 0!n**. Pozwala zapytać, co jest przedmiotem opisu, skąd pochodzi wiedza, jakie właściwości odczytano, czego nie wiadomo, jak obiekt może się łączyć z innymi i jaki dowód potwierdzi realizację jego zadania. W tym znaczeniu artefakt jest zarazem jednostką organizacji wiedzy i potencjalną jednostką kompozycji. Jego wykonawcza użyteczność zależy od dodatkowych, sprawdzalnych powiązań.

Praca powstała przez zestawienie przekazanego pakietu wiedzy `lion-rag30-v1.5-r1` z lokalnym kodem LION Polyhedral Studio R6, wersja 0.6.0. Sprawdzono integralność 29 plików wymienionych w manifeście pakietu wiedzy. Jego zadeklarowany cutoff przypada na 4 października 2026. Analiza kodu dotyczy przypiętego wydania R6, a nie obserwacji bieżącego wdrożenia całej federacji. Wnioski o idei 0!n są syntezą interpretacyjną; opisy funkcji wskazują istniejące mechanizmy; uogólnienia fabryki i nowe scenariusze tworzenia pozostają projektami, gdy implementacja ich jeszcze nie realizuje. [1,2]

Taki podział jest potrzebny, ponieważ ta sama nazwa występuje w kilku porządkach. Możemy mieć zapisany model workera, jego specyfikację, pliki implementacji i uruchomiony proces. Każdy jest uprawnionym obiektem analizy, lecz każdy odpowiada na inne pytanie. Umiejętność utrzymania tych różnic stanowi podstawę samorozwoju: system musi wiedzieć, czy zmienił deklarację, projekt, kod czy rzeczywiste zachowanie.

1. Obiekt istnieje w ramie pytań i odniesień

Obiekt nie musi rozpoczynać swojego życia w 0!n jako instancja klasy programistycznej. Już pytanie „co w tym procesie odpowiada za weryfikację wyniku?” wydziela możliwy przedmiot architektoniczny. Może się okazać, że jest nim jedna funkcja, kilka modułów, osobna usługa albo brakująca zdolność. Pierwszą operacją jest zatem ustalenie granicy: co dokładnie modelujemy i jaki problem uzasadnia taką jednostkę.

Granica ta jest zależna od celu, ale powinna być jawna. Ten sam plik może być analizowany jako ciąg bajtów, moduł Pythona, zbiór deklaracji, nośnik kontraktu albo składnik dystrybucji. Są to różne perspektywy na powiązane przedmioty. Przy badaniu integralności interesują nas bajty i skrót. Przy rekonstrukcji architektury interesują nas symbole i importy. Przy uruchamianiu programu potrzebny jest jeszcze faktyczny composition root, konfiguracja i środowisko wykonania.

Nie należy więc ustanawiać jednego niezmiennego podziału, w którym każdy plik musi być jednym artefaktem, każda klasa jednym wnętrzem, a każdy katalog jednym poziomem abstrakcji. Granice fizycznego zapisu i odpowiedzialności logicznej mogą się pokrywać, lecz nie muszą. Funkcja pomocnicza bywa częścią jednej odpowiedzialności rozproszonej po kilku plikach. Jeden moduł może zawierać kilka niezależnych odpowiedzialności. Model powinien pozwalać te sytuacje opisać bez fałszowania rzeczywistej lokalizacji kodu.

Wprowadźmy pomocniczy zapis idei, który nie jest nowym formatem implementacyjnym:

```
obiekt rozpatrywany = przedmiot + granica + pytanie + kontekst

opis artefaktowy = tożsamość + właściwości + interfejsy
                 + relacje + pochodzenie + stan wiedzy
```

Pierwsza linia określa, na co patrzymy. Druga określa, jak uczynić to dostępne dla pozostałych części systemu. Opis może obejmować obiekt istniejący albo zamierzony. W drugim przypadku już istnieje specyfikacja, natomiast opisywana realizacja może dopiero powstać. Odczytanie specyfikacji potwierdza dostęp do jej treści; nie potwierdza wykonania opisanej funkcji.

Relacyjny charakter takiej architektury ma znaczenie praktyczne. Nazwanie obiektu „analizatorem” niewiele mówi bez informacji, co analizuje, dla kogo, według jakiego kontraktu i z jaką granicą błędu. Te same bajty mogą zajmować różne miejsca odpowiedzialności, a ta sama odpowiedzialność może mieć kolejne implementacje. 0!n powinno zachowywać zarówno identyfikację przedmiotu, jak i jego zmienne ulokowanie w systemie.

Nie wynika stąd teza metafizyczna, że nic nie istnieje poza relacjami. Wystarcza twierdzenie architektoniczne: **znaczenie obiektu dla aplikacji powstaje w określonej ramie i w określonych związkach**. Pozwala to traktować jako przedmiot modelowania także relację, twierdzenie o relacji czy test tego twierdzenia. Obecny format sceny ma jednak określony model końców połączeń; ogólnego rachunku relacji między relacjami nie należy dopisywać do jego możliwości bez dodatkowego kontraktu. [3,4]

2. Kwalifikacja jest wielowymiarowym rozpoznaniem

Zakwalifikowanie obiektu oznacza udzielenie kilku niezależnych odpowiedzi. Trzeba ustalić rodzaj dostępnej reprezentacji, sposób jej odczytu, typy właściwości, rolę w kompozycji, źródło twierdzeń, aktualność oraz stopień materializacji. Jedna etykieta, na przykład „gotowy”, nie zachowuje tych różnic.

Na poziomie dostępu pytamy, czy mamy tekst dokumentu, wartość JSON, rekord katalogu, źródło Pythona albo obserwację konkretnej usługi. Na poziomie struktury pytamy o pola, symbole, parametry i zależności. Na poziomie znaczenia pytamy o cel, wymagania, założenia i konsekwencje. Na poziomie dowodu pytamy, czy twierdzenie pochodzi z deklaracji autora, statycznego odczytu, testu, symulacji czy obserwacji wykonania. Każda odpowiedź ma własny zakres ważności.

Przykładowo plik zawierający funkcję `validate_report` pozwala stwierdzić, że w odczytanej wersji źródła istnieje taka deklaracja. Jej nazwa sugeruje zadanie, ale nie rozstrzyga, czy sprawdza wszystkie wymagane warunki. Test może potwierdzić zachowanie na określonych danych. Uruchomiony proces może wykonywać inną wersję pliku. Kwalifikacja powinna umożliwiać zapisanie wszystkich tych faktów jednocześnie.

W rdzeniu R6 wybór czytnika jest jawny. `ReaderRegistry` udostępnia trzy wbudowane rodzaje: `json_value`, `document` oraz `catalog_node`. Pierwszy rozpoznaje strukturę dostarczonej wartości. Drugi potwierdza dostęp do tekstu, jego długość w UTF-8 i skrót. Trzeci rozpoznaje rekord katalogu z wymaganymi polami identyfikacji. Wskazane w rekordzie źródło pozostaje referencją, dopóki odrębny mechanizm go nie sprawdzi. [5]

Z tego powodu słowo „dowolny” wymaga dwóch znaczeń. W idei 0!n dowolny przedmiot może stać się przedmiotem modelowania, jeśli potrafimy zbudować jego adekwatną reprezentację i regułę odczytu. W implementacji konkretna wersja obsługuje określoną rodzinę wejść. Jeżeli dla nowego rodzaju przedmiotu nie ma czytnika, wynikiem powinien być rozpoznany brak zdolności albo projekt nowego adaptera. System nie zyskuje wiedzy przez samo przydzielenie obiektowi nazwy.

Kwalifikacja semantyczna także wymaga pochodzenia. Jeżeli model językowy proponuje, że funkcja realizuje rolę „weryfikacja dowodów”, otrzymujemy hipotezę o odpowiedzialności. Można ją porównać z kodem, testami i kontraktem. Wspólny standard pozwala tę propozycję przechować, skierować do oceny i później poprawić. Sam zapis nazwy roli nie stanowi dowodu trafności klasyfikacji.

Wartości nieznane są częścią tej logiki. Brak pola, wartość `null`, wartość zero, `false` i niewłaściwy typ oznaczają odmienne sytuacje. Dla parametru „liczba błędów” zero może być poprawnym wynikiem pomiaru. Nieznana liczba błędów wymaga pozostawienia niewiedzy. Zastąpienie jej zerem zmieniłoby stan poznawczy systemu, a następnie mogłoby zmienić decyzję.

3. Ośmiościan ścięty jako wspólna gramatyka interfejsu

Ośmiościan ścięty dostarcza w 0!n wspólnego układu czternastu powierzchni. W przyjętym profilu sześć ścian kwadratowych odpowiada rolom podstawowym, a osiem sześciokątnych rolom funkcjonalnym. Związek nazw ról z geometrią jest decyzją architektoniczną. Geometria umożliwia konsekwentne adresowanie, nawigację oraz przedstawienie granicy obiektu; szczegółowe znaczenie wynika z przyjętego kontraktu. [6]

Sześć ról podstawowych porządkuje warunki rozpoznawalności i rozliczalności. `identity` odpowiada na pytanie, który przedmiot opisujemy. `contract` określa obietnice i ograniczenia. `boundary` wyznacza zakres odpowiedzialności. `provenance` wiąże opis ze źródłami. `lifecycle` pozwala odróżniać stany i następców. `observability` wskazuje, jak można sprawdzać stan i wynik.

Osiem ról funkcjonalnych porządkuje udział w większym układzie. `input` i `output` opisują wymianę danych. `context` zachowuje ramę interpretacji. `knowledge` dotyczy używanej lub wytwarzanej wiedzy. `cognition` może opisywać funkcje poznawcze. `composition` wskazuje

sposób składania części. `routing` dotyczy kierowania współpracą. `evolution` opisuje zmianę, pochodzenie generacji i warunki rozwoju.

Ściana jest adresem rodziny pytań i możliwych interfejsów. Nie każda musi mieć aktywne porty. Dokument badawczy może mieć rozbudowane pochodzenie i wiedzę, a nie mieć żadnego interfejsu wywołania. Funkcja obliczeniowa może mieć konkretne wejścia i wyjście, lecz nie realizować poznawania ani ewolucji. Wspólny szkielet pozwala porównywać takie obiekty bez dopisywania im fikcyjnych zdolności.

Trzeba przy tym rozróżnić ścianę, kafel i port. Ściana wyznacza rolę. Kafel konkretyzuje deklarację na tej powierzchni. Port wskazuje miejsce wymiany i może być związany z nazwanym parametrem. Obecna projekcja pojedynczego odczytu tworzy po jednym kaflu dla każdej z czternastu ścian, natomiast szerszy model sceny dopuszcza bogatszy opis. Typ, kierunek, jednostka i schemat portu pomagają określić, co wolno połączyć.

Dziewięć osi `T,S,R,E,I,F,A,P,D` opisuje inną przestrzeń. W R6 są to jawnie przekazywane współrzędne semantycznego kontekstu. Nie ma odwzorowania jednej osi na jedną ścianę. W historycznym opisie HMK litera A dotyczy abstrakcji; nie nadaje uprawnień. Wypełnienie osi albo narysowanie brył blisko siebie nie uruchamia komunikacji. Znaczenie położenia, odległości i siły związku musi być zdefiniowane przez konkretny profil lub algorytm, jeżeli ma wpływać na proces. [6,7]

Geometria staje się użyteczna wykonawczo wtedy, gdy określony adres powierzchni prowadzi przez jawne definicje do zgodnego interfejsu i właściwego wykonawcy. Sama scena może przedstawiać te same adresy człowiekowi. Dzięki temu obraz i mechanizm mają wspólną referencję, bez przenoszenia własności kamery na semantykę systemu.

4. Automatyczna artefaktyzacja: rzeczywisty algorytm R6

Publiczny rdzeń udostępnia łańcuch `observe_value`, `Observation`, `artifactize` oraz `project_from_readings`. To deterministyczny mechanizm budowy reprezentacji z jawnych danych. Jego automatyczność polega na zastosowaniu ustalonej reguły, obliczeniu identyfikatorów, wypełnieniu parametrów i utworzeniu zgodnego modelu. Nie oznacza automatycznego wyczerpania całego znaczenia obiektu. [5,8]

Na wejściu znajdują się wartość, `identity_ref`, rodzaj czytnika, profil oraz kontekst. Tożsamość logiczną dostarcza wywołujący. Rodzaj czytnika wybiera znaną metodę analizy. Profil określa, które fragmenty danych odczytać jako zawartość ścian i parametry oraz jakie zadeklarować porty. Kontekst może podać nazwę, opis, źródła, osie semantyczne i odniesienie do rodzica. Te deklaracje pozostają odróżnialne od faktów wyliczonych przez czytnik.

Najpierw walidowana jest konfiguracja. Niewłaściwy profil, nieobsługiwane pola kontekstu czy błędny rejestr powodują błąd konfiguracji `ReadingError`. Potem ograniczana i kanonizowana jest sama wartość. Mechanizm `canonical` dopuszcza zwykłe, skończone drzewa JSON: słowniki, listy, teksty, liczby, wartości logiczne i `None`. Odrzuca między innymi cykle, liczby nieskończone i arbitralne instancje klas. Obowiązują również limity rozmiaru i zagnieżdżenia. [9]

To ma bezpośrednie znaczenie na poziomie Pythona. Próba „zajrzenia do dowolnego obiektu” przez `repr`, `getattr`, iterację lub niekontrolowaną serializację może uruchomić kod metod użytkownika. Rdzeń nie traktuje takiej introspekcji jako neutralnego odczytu. Integrator może jawnie zarejestrować zaufany adapter hosta, lecz jego zachowanie wymaga osobnej oceny. Dane żądania nie wybierają samodzielnie modułu do importu ani funkcji do wykonania.

Jeżeli wartość nie jest obsługiwana albo brakuje czytnika danego rodzaju, powstaje obserwacja ze statusem `UNSUPPORTED` i diagnostyką. Nie jest to dowód usunięcia obiektu. Oznacza brak poprawnego odczytu w danej konfiguracji. Rozróżnienie błędu konfiguracji i nieobsługiwanego payloadu pozwala skierować poprawkę we właściwe miejsce.

Po pomyślnym odczycie obliczany jest skrót treści. Czytnik zwraca ograniczony zestaw faktów zgodny ze swoim zadaniem. Profil stosuje wskaźniki JSON Pointer do podanych danych. Jeżeli wskaźnik trafia w istniejące pole, odpowiednia powierzchnia uzyskuje status odczytanej wartości. Jeżeli pole nie istnieje, pozostaje `UNKNOWN`. Samo trafienie wskaźnika nie sprawdza prawdziwości zdania zapisanego w tym polu.

Powstający rekord `Observation` zachowuje payload, metodę odczytu, pełną definicję profilu, kontekst, kwalifikację, skrót treści i identyfikator całego odczytu. Jego wewnętrzna reprezentacja jest niezmiennym zapisem bajtów. Metoda `to_dict()` udostępnia nową kopię danych. Przy wczytywaniu `Observation.from_dict()` sprawdza skrót oraz odtwarza wynik przy użyciu właściwego czytnika. Odtwarzalność derywacji jest silniejszą kontrolą niż zgodność samego hasha dowolnie przygotowanego rekordu.

Następnie `artifactize()` ponownie sprawdza odczyt. Nie projektuje odmowy obsługi ani treści sprzecznej z zadeklarowanym pinem jako poprawnego zaobserwowanego artefaktu. Wyprowadza stabilne ID artefaktu z `identity_ref`, tworzy parametry i czternaście adresowanych kafli, zachowuje źródła oraz metadane. Kończy normalizacją i walidacją według istniejącego modelu Studio v2, zamiast tworzyć drugi, równoległy język artefaktów.

Odczytany parametr ma kilka możliwych kwalifikacji. Wartość zgodna z deklarowanym typem i różna od `null` otrzymuje `READ`, także gdy jest zerem, `false` albo pustym tekstem. Jawne `null` otrzymuje `OBSERVED_NULL`. Brak ścieżki daje `UNKNOWN`, a znaleziona wartość niewłaściwego typu daje `TYPE_MISMATCH`. W tych dwóch ostatnich przypadkach wartość w projekcji może być `None`, ale metadane zachowują przyczynę. Jednostka pochodzi z deklaracji profilu; czytnik nie przeprowadza przez to automatycznie pomiaru ani analizy wymiarowej.

Artefakt uzyskuje kwalifikację odczytanej reprezentacji, a jednocześnie pozostaje projektem: `materialization=DESIGN`, `runtime_binding=NOT_BOUND`, `source_execution=NONE`. Odpowiada to stanowi „mamy sprawdzalny opis odczytanych danych”. Jest to wystarczające do modelowania i dalszej analizy, natomiast wykonanie wymaga następnych powiązań.

`project_from_readings()` składa wiele takich odczytów we wspólny projekt. Rozwiązuje rodziców wskazanych przez `parent_identity_ref`; nie zgaduje brakującego rodzica. Przyjmuje jawne rekordy relacji. Sortuje i rozmieszcza obiekty w sposób deterministyczny,

lecz ta pozycja służy prezentacji. Podobna nazwa, identyczna wartość ani mała odległość nie tworzą same zależności przepływu.

5. Przykład: odczyt wartości i świadomie zaprojektowany profil

Rozważmy niewielką reprezentację wyniku: nazwa miernika oraz liczba wykrytych problemów. W przykładzie analizujemy dostarczony rekord JSON. Nie odczytujemy urządzenia ani procesu wykonującego pomiar. Profil wskazuje, że nazwa ma zasilić powierzchnię kontekstu, liczba ma stać się parametrem `count`, a parametr ma mieć zadeklarowany port wyjściowy. Nazwy `example.*` są lokalnymi identyfikatorami przykładu.

```
from lion_studio.artifact_core import (
    ReadingProfile, observe_value, artifactize,
)

profile = ReadingProfile.from_dict({
    "schema": "lion.reading-profile/v1",
    "id": "example.issue-count",
    "version": "1.0",
    "geometry": "truncated-octahedron-14",
    "face_bindings": {
        "context": {"pointer": "/name"},
    },
    "parameter_bindings": {
        "count": {
            "pointer": "/count",
            "type": "integer",
            "unit": "",
            "schema_ref": "example.issue-count/v1",
        },
    },
    "ports": [{
        "id": "count-out",
        "face": "output",
        "direction": "out",
        "parameter": "count",
        "contract_ref": "example.issue-count/v1",
    }],
})

reading = observe_value(
    {"name": "Kontrola raportu", "count": 0},
    "example:report-check:42",
    profile=profile,
    context={"name": "Wynik kontroli raportu"},
)
artifact = artifactize(reading)

assert reading.status == "READ"
assert len(artifact["tiles"]) == 14
assert artifact["parameters"]["count"]["value"] == 0
assert artifact["axes"]["materialization"] == "DESIGN"
assert artifact["metadata"]["artifact_core"][
    "runtime_binding"
] == "NOT_BOUND"
```

Ten przykład ujawnia rolę autora profilu. Algorytm nie odkrył samodzielnie, że `/count` ma oznaczać liczbę problemów. Tę interpretację ustanowiono w regule odczytu. Algorytm wykonał regułę, sprawdził typ i zachował związek z treścią. W ten sposób rozdzielono projekt semantyki od powtarzalnego zastosowania tej semantyki do danych.

Zachowanie zera jest istotne: jest to rozpoznana liczba, a nie brak. Pusta jednostka oznacza brak zadeklarowanej jednostki w profilu. Port wyjściowy stanowi opis kontraktu. Nie został dzięki niemu odnaleziony proces pomiarowy, przyznane uprawnienie ani wykonany pomiar. Gdy dołączymy nowego konsumenta, osobno określimy jego zgodność z tym wyjściem.

Możemy wykonać kontrpróby bez zmiany mechanizmu. Rekord `{"name": "Kontrola raportu"}` da dla parametru `UNKNOWN`. Rekord z `"count": null` da `OBSERVED_NULL`. Rekord z `"count": "zero"` da `TYPE_MISMATCH`. Pełny odczyt reprezentacji nadal może mieć status `READ`, ponieważ sam rekord jest czytelną wartością JSON. Kwalifikacja całości i kwalifikacja jednego parametru dotyczą innych poziomów.

Zmiana danych nie musi zmieniać tożsamości obiektu. Zmiana `count` z 0 na 2 dla tej samej `identity_ref` tworzy inną treść odczytu tej samej jednostki logicznej. Zmiana profilu może natomiast zmienić interpretację niezmienionej treści. To właśnie dlatego skrót payloadu, skrót profilu i identyfikator całej obserwacji muszą pozostać rozróżnialne.

6. Rekonstrukcja z kodu: co da się odzyskać automatycznie

Kod istniejącej aplikacji jest szczególnym obiektem, ponieważ jednocześnie stanowi tekst, strukturę składniową i potencjalny mechanizm wykonawczy. Rekonstrukcja w 0!n rozpoczyna się od określenia źródła: lokalnego drzewa plików albo konkretnego refu Git. Trzeba zachować ścieżkę, wersję i hash odczytanych bajtów. Lokalna niezapisana zmiana, commit i plik uruchomiony przez interpreter mogą reprezentować różne stany tego samego projektu.

Mechanizm `read_self_model()` odczytuje własne źródła Studio przez zdefiniowany deskryptor. Korzysta z indeksowania i analizy składniowej. AST pozwala wyodrębnić moduły, klasy, funkcje, sygnatury i część kandydatów wywołań. Analizowane źródło nie musi być importowane ani uruchamiane, aby odczytać deklarację. Dzięki temu modeler może rekonstruować swoją strukturę bez wykonywania znalezionych funkcji. [10]

Z funkcji można wyprowadzić wstępny opis wejść, z modułu jego zawierane deklaracje, z importu kandydacką zależność źródłową. Pozostają jednak granice. Dynamiczny import, podmiana obiektu, dekorator, konfiguracja albo polimorficzne wywołanie mogą zmieniać rzeczywistą ścieżkę wykonania. Statyczny kandydat wywołania nie jest obserwacją, że funkcja została wywołana w badanej misji.

Również adnotacja typu nie stanowi dowodu kontraktu zachowania. `-> int` oznacza określony zapis w źródle; program może naruszyć tę deklarację. Typ „integer” w modelu portu jest dalszą interpretacją. W badanym R6 występuje przy tym konkretna rozbieżność adapterów: importer zapisuje `return_annotation`, a budowniczy portu wyjściowego szuka pola `returns`. W rezultacie odczytana adnotacja zwrotna może pozostać w metadanych,

podczas gdy typ portu ma wartość `any`. Ten przypadek pokazuje, dlaczego sam analizator również wymaga testu zgodności producenta i konsumenta. [10]

Automatycznie odzyskana scena jest projekcją ograniczonego zakresu. Jeżeli importer pobrał tylko część plików lub wyświetla tylko część węzłów, brak obiektu na scenie nie dowodzi jego nieistnienia w repozytorium. Przejście do wnętrza powinno zachowywać źródło i informację o pokryciu. Inaczej użytkownik nie odróżni pustego wnętrza od wnętrza jeszcze niezaindeksowanego.

Pełna rekonstrukcja odpowiedzialności wymaga zestawienia kilku rodzajów śladów. Deklaracja, miejsce użycia, kontrakt, test i obserwacja wykonania mogą się wzajemnie uzupełniać. Dokumentacja może wskazywać cel, a kod częściowo go realizować. Test może wykazać kontrprzykład. Model powinien przechować to jako rozbieżność wymagającą pracy, zamiast wymuszać jeden zgodny obraz przez przemianowanie elementów.

Z perspektywy przebudowy starszego LION jest to fundamentalne. Początkowy model powinien zachowywać to, co rzeczywiście odczytano, a następnie pozwalać wyprowadzać docelowe artefakty odpowiedzialności. Relacja „ten moduł realizuje część tego kontraktu” może być dokładniejsza niż utożsamienie modułu z kontraktem. Dopiero plan refaktoryzacji wskazuje, jakie pliki, interfejsy i testy powinny się zmienić, aby nowe granice logiczne otrzymały właściwą realizację.

7. Tworzenie artefaktu: od potrzeby do sprawdzalnego kandydata

Gdy chcemy stworzyć artefakt, punktem wyjścia jest potrzeba i obraz stanu, w którym ta potrzeba nie jest jeszcze spełniona. To odwraca kierunek poznania. Przy rekonstrukcji przechodzimy od dostępnej reprezentacji do opisu jej roli. Przy projektowaniu przechodzimy od oczekiwanej roli do specyfikacji możliwej realizacji. Obie drogi mogą spotkać się w tym samym standardzie modelu, zachowując odmienne pochodzenie twierdzeń.

W linii Bean Factory dokumentacja rozróżnia `Gap`, `CapabilityNeed`, `BeanSpec`, `BeanCandidate` i `BeanInstance`. Są to rozwojowe pojęcia federacji, a nie zamienne nazwy każdego węzła Studio. Luka wymaga odniesienia do celu i odczytanego stanu. Potrzeba zdolności opisuje wymaganie. Specyfikacja ustala oczekiwane wejścia, wyjścia, ograniczenia i kryteria. Kandydat proponuje realizację. Instancja ma odrębny kontekst faktycznego użycia. [11]

Pierwszą decyzją konstrukcyjną powinno być sprawdzenie możliwości ponownego użycia. Jeżeli w systemie istnieje zgodny komponent, zadaniem może być jego złożenie lub dostosowanie. Samo podobieństwo nazwy nie rozstrzyga zgodności: trzeba porównać kontrakt danych, wersję, zakres, warunki błędu i zachowanie. Tworzenie nowego kodu bez tego kroku powiększa liczbę bytów, ale może nie powiększyć zdolności systemu.

Rozważmy potrzebę utworzenia walidatora raportów JSON. Artefakt zamierzony powinien określać, jaki dokument przyjmuje, jakie reguły sprawdza i jaki wynik zwraca. Należy ustalić, czy brakuje pola, czy pole ma błędną wartość, czy wejście jest nieczytelne. Potrzebne są przykłady poprawne i kontrprzykłady. Dopiero te decyzje uzasadniają porty, parametry i części wewnętrzne.

Możemy zaprojektować trzy części: czytnik danych, walidator reguł i budowniczy wyniku. Wspólna granica artefaktu eksponuje kontrakt dla konsumenta. Wnętrze może mieć bogatszy graf przepływu, ale jego szczegóły nie powinny być wymagane do każdego użycia zewnętrznego. Zmiana wnętrza przy zachowaniu kontraktu jest innym rodzajem ewolucji niż zmiana samego kontraktu.

Na poziomie kodu taki zamiar można już zapisać jako projekt v2. Normalizacja dopisuje pola prezentacji i wartości domyślne; walidacja bada strukturę. Poniższy przykład tworzy opis przyszłego komponentu. Nie wymaga jeszcze gotowej implementacji walidatora. Wszystkie czternaście powierzchni otrzymuje adresy, natomiast niewypełnione role pozostają deklaratywnym miejscem do dalszego projektu.

```
from lion_studio.model import (
    FACES, normalize_workspace, validate_workspace,
)

project = normalize_workspace({
    "project_id": "example:report-validator-design",
    "name": "Projekt walidatora raportów",
    "artifacts": [{
        "id": "validator-design",
        "name": "Walidator raportów JSON",
        "kind": "proposed_component",
        "description": "Kandydat do realizacji kontraktu",
        "tiles": [
            {"id": "face:" + face, "face": face,
             "role": face, "ports": []}
            for face in FACES
        ],
    }],
})

errors = [
    issue for issue in validate_workspace(project)
    if issue["severity"] == "error"
]
assert not errors
assert project["artifacts"][0]["axes"][
    "evidence"
] == "DECLARED"
```

Poprawność strukturalna tego projektu nie oznacza, że walidator spełnia wymaganie. Nie ma jeszcze jego reguł, zachowania ani dowodu testowego. Przykład pokazuje za to ważną własność: artefakt zamierzony jest dopuszczalnym obiektem pracy, jeżeli jego stan jest jawny. System może porównywać warianty jego konstrukcji, dopisywać kontrakty i planować materializację bez udawania istniejącej zdolności.

Dalsza praca polega na związaniu projektu z konkretnymi bajtami realizacji, uruchomieniu odpowiednich testów i odczycie wyniku. Kiedy implementacja powstanie, sama również może zostać odczytana jako obiekt. Porównanie projektu z rekonstrukcją kodu odpowie, czy realizacja zachowała założone granice i kontrakty. W ten sposób projektowanie i odczyt zamykają wspólną pętlę.

8. Jak części tworzą spójny artefakt i jak artefakt tworzy proces

Spójność artefaktu nie wynika z samego zamknięcia wielu obiektów w jednej bryle. Potrzebny jest powód ich wspólnej granicy. Może nim być kontrakt, odpowiedzialność, cykl życia albo kontrolowany zestaw warunków wykonania. Zawieranie pozwala określić wnętrze, lecz dopiero zgodność współpracy wyjaśnia, dlaczego części tworzą użyteczną całość.

Trzeba rozdzielić kilka relacji. `parent_id` wyznacza strukturę zawierania. Relacja semantyczna może mówić o zależności, uzasadnieniu albo podobieństwie. Przepływ wiąże konkretne porty i parametry. Ekspozycja określa przejście przez granicę. Wspólna przynależność do grupy może służyć analizie i widokowi. Żadna z tych informacji nie powinna być automatycznie przepisywana na inną tylko dlatego, że jest przedstawiana linią.

Modelowanie obiektu względem artefaktu oznacza więc odpowiedź na pytanie o rodzaj przynależności. Funkcja może być wewnętrzną implementacją, zewnętrzną zależnością, źródłem dowodu lub kandydatem do zastąpienia części. Hipoteza może opisywać zachowanie artefaktu, a test ją podważać. Umieszczenie wszystkich tych przedmiotów jako jednakowych dzieci zacierałoby różnice potrzebne później do zmiany systemu.

Przepływ wymaga kontraktu końców. Port wyjściowy i wejściowy mają kierunek, typ i wskazanie schematu. Wiązanie określa, który parametr zasila który i czy zachodzi dozwolona transformacja. Rewizje końców wiążą relację z konkretnym stanem interfejsów. Gdy interfejs się zmieni, wcześniejsze połączenie może wymagać ponownej oceny. Wspólna nazwa parametru nie zastępuje tej kontroli.

W R6 `compile_process()` bierze pod uwagę relacje `dataflow` i `exposes`, podczas gdy relacje opisowe pozostają opisowe. Sprawdza strukturę, przypięcia rewizji, bramki, wiązania, jawne typy i schematy portów oraz konkurujących pisarzy. Bezpośredni przepływ między różnymi rodzicami jest traktowany jako przekroczenie granicy wymagające właściwego rozwiązania. Cykle potrzebują skończonych limitów sprzężenia. Wynik kompilacji jest harmonogramem projektowym, bez wywołania providerów. [12]

Wartość bramki może być prawdziwa, fałszywa lub nieznana. Fałsz wyłącza relację w badanym scenariuszu. Niewiedza zatrzymuje uznanie jej za gotowy przepływ. To pozwala utrzymać różnicę między „warunek nie zachodzi” i „nie mamy danych, by go rozstrzygnąć”. W większym systemie właśnie ta różnica chroni przed budowaniem procesu na milcząco dopisanych założeniach.

Spójny proces wymaga ponadto celu, kryterium ukończenia i sposobu sprawdzenia wyniku. Statyczny graf zgodnych połączeń może opisywać poprawną wymianę, a mimo to nie realizować potrzeby operatora. Dla walidatora raportów trzeba wykazać, że wykrywa wymagane naruszenia i nie odrzuca dopuszczonych przypadków. Dopiero wówczas kompozycja jest uzasadniona zarówno strukturalnie, jak i użytkowo.

9. Broker, modele i próg materialnego działania

Broker jest niezależnym elementem współpracy. Ma własne interfejsy, odpowiedzialności, stan i możliwość odrębnego opisu artefaktowego. Modeler przedstawia i konstruuje modele. Broker kieruje żądania do właściwych funkcji i usług zgodnie z dostępnymi definicjami. To rozdzielenie pozwala zmieniać sposób prezentacji i planowania bez czynienia sceny 3D źródłem dowolnego wykonania.

`ArtifactService` jest adapterem operacji artefaktowych wywoływanym przez wybrany broker routujący. Udostępnia między innymi odczyt, projekcję, historię, uzgadnianie odczytów, kompilację procesu, ewolucję i samoopis. CoreAPI może wyprowadzać rejestr poleceń z jawnych definicji obiektów i interfejsów. Obowiązuje jednak związek z konkretnym providerem: sama obecność nazwy funkcji w deskryptorze nie tworzy działającej usługi. [13]

W szerszej idei 0!n broker umożliwia łączenie analizy deterministycznej z modelami probabilistycznymi. Parser może odczytać składnię, model może zaproponować odpowiedzialność, narzędzie może sprawdzić kontrprzykład, a komparator może ocenić kandydata. Wyniki tych etapów powinny zachować odrębne kwalifikacje. Propozycja modelu może uruchomić dalsze pytanie lub test, jeżeli istnieje właściwy wykonawca i zakres; nie staje się przez to obserwacją świata.

Warto odróżnić materializację opisu od materializacji implementacji. Zapis projektu, eksport paczki artefaktu albo wygenerowanie dokumentacji tworzą rzeczywiste bajty, lecz mogą wciąż reprezentować projekt. Zapis pliku z nowym kodem jest dalszym efektem. Uruchomienie tego kodu w procesie i wdrożenie go do konkretnego środowiska są kolejnymi zdarzeniami.

Obecny broker zmian plikowych Studio działa w swoim ograniczonym zakresie katalogów roboczych. Nie oznacza to automatycznego refaktoryzowania dowolnego checkoutu operatora. Plan określa oczekiwane wejściowe bajty i zamierzoną zmianę; wykonanie powinno zwrócić wynik, a readback porównać stan z oczekiwaniem. Receipt wykonawcy i odczyt wyniku mają uzupełniające role. Zapisany status sukcesu sam nie zastępuje sprawdzenia pliku. [13]

Dokumenty federacji dodatkowo rozróżniają propozycję, decyzję dopuszczającą określony zakres, admission runtime, efekt, obserwację i uzgodnione ukończenie. To różne odpowiedzialności. Artefakt opisujący model albo brokera może obejmować kontrakty tych przejść, lecz nie nadaje sobie przez to prawa do ich samodzielnego rozszerzenia. System może budować kandydata nowego control plane; jego własna ocena kandydata nie wystarcza do wdrożenia nowej władzy wykonawczej. [14]

10. Stały odczyt, pamięć i pętla ewolucyjna

Sformułowanie „nie istnieje stały zapis, istnieje stały odczyt” można technicznie rozumieć jako pierwszeństwo wersjonowanej reguły interpretacji nad raz na zawsze zamrożonym opisem świata. Obiekt się zmienia, a system potrafi ponownie zastosować określoną ramę i porównać wyniki. Nie oznacza to rezygnacji z trwałych zapisów ani ciągłego skanowania wszystkiego bez ograniczeń.

Pamięć odczytów jest potrzebna, aby stwierdzić, co się zmieniło. Tożsamość logiczna odpowiada na pytanie „który obiekt?”, digest treści na „jakie dane?”, wersja i digest profilu na „według jakiej reguły?”, lokalizator na „gdzie?”, a identyfikator odczytu wiąże pełny rekord. Powtórzenie identycznego wejścia z identycznym profilem i kontekstem daje ten sam deterministyczny odczyt. Zmiana profilu lub kontekstu może zmienić jego identyfikator mimo niezmienionego payloadu.

`ReadingStore` zachowuje historię i aktualną zaakceptowaną głowę danej tożsamości. Może rozpoznać `ACCEPTED`, `UNCHANGED` i `STALE` w odniesieniu do podanych oczekiwań. Aktualna głowa jest aktualnym wpisem tego magazynu, a nie gwarancją świeżości zewnętrznego pliku. Jeżeli źródło zostało później zmienione, potrzebna jest kolejna obserwacja. W zbiorczym przyjęciu odczytów należy też odróżnić aktualizację poszczególnych głów od gotowości całej wspólnej projekcji. [13]

Ewolucja rozpoczyna się od różnicy między wymaganiem a stanem, który umiemy uzasadnić. Kandydat ma zmniejszyć tę różnicę. Test lub falsyfikator szuka sytuacji, w której nie spełnia on celu. Kontrprzykład wraca jako wejście poprawionej generacji. Gdy pojawia się rzeczywista zmiana, jej wynik zostaje ponownie odczytany i związany z projektem. Historia zachowuje również odrzucone warianty, ponieważ wyjaśniają one, dlaczego wybrano następny.

Pomocniczo można ten obieg zapisać następująco. Jest to opis architektury, nie kod uruchamiający misję:

```
potrzeba + odczytany stan -> luka i kryteria
luka -> reuse albo specyfikacja nowego kandydata
kandydat -> analiza, testy i kontrprzykłady
wynik oceny -> wybór albo poprawiona generacja
wybrany plan + właściwy wykonawca -> ograniczony efekt
odczyt efektu -> uzgodniony wynik i następna obserwacja
```

W implementacji R6 `evolve()` realizuje ograniczoną pętlę projektową: przyjmuje bazę, jawne kryteria i kandydatów albo skonfigurowany algorytm. Liczba generacji jest ograniczona do szesnastu. Kryteria mają własny digest, a pojedyncza generacja nie może po cichu zmienić zasad oceny. Domyślny tryb wymaga dostarczonego readbacku; opcjonalny tryb symulowany oznacza syntetyczny wynik jako symulację. Zewnętrzny efekt i uprawnienia pozostają poza tą funkcją. [12]

Pełna pętla samorozwoju jest szerszym wymaganiem. Potrzebuje rzeczywistych providerów, kontraktów, wykonawców, obserwatorów i recovery. Zdolność do wygenerowania modelu oraz przetestowania dwóch rodzin kandydatów ma określony

zakres; nie dowodzi dowolnej generatywności. Dzięki jawnemu opisowi tego zakresu można jednak planować następne przyrosty bez odbierania wartości już działającym funkcjom.

Sam modeler może być przedmiotem takiej pętli. `run_studio.py`, serwer, model danych, czytniki, broker, interfejs i testy dają się opisać w tej samej ramie co analizowana aplikacja. Zmiana czytnika lub kryterium sukcesu wymaga jednak osobnej oceny, ponieważ wpływa na sposób rozpoznawania późniejszych wyników. Samoopis oznacza wspólną procedurę opisu; nie ustanawia bezbłędności ani samoczynnego zatwierdzania zmian.

Zakończenie: wspólna forma dla poznawania i konstruowania

Obiekt w 0!n jest przedmiotem rozpoznawalnym w jawnej ramie. Artefakt nadaje mu postać, w której można zachować tożsamość, właściwości, interfejsy, pochodzenie i relacje. Dzięki temu plik, idea, wynik badania i proces mogą uczestniczyć we wspólnym modelu, zachowując różnice rodzaju oraz zakresu wiedzy.

Automatyczna analiza ustanawia związek między dostępną reprezentacją a jej opisem. W obecnym rdzeniu robi to przez jawny czytnik, profil, obserwację i projekcję do modelu v2. Tworzenie zamierzone ustanawia związek między potrzebą a specyfikacją i kandydatem. Rozwój następuje wtedy, gdy obie drogi spotykają się w sprawdzalnej realizacji: możemy porównać to, co miało powstać, z tym, co odczytano i co faktycznie działa.

Właśnie tutaj znajduje się praktyczny sens mozaikowości. Wspólna forma umożliwia składanie różnorodnych części, a zachowane granice pozwalają je badać i zmieniać. Czternaście powierzchni daje gramatykę interfejsu; relacje nadają architekturze strukturę; protokoły zachowują kontekst współpracy; broker wiąże żądania z dostępnymi funkcjami; pamięć i testy umożliwiają odtworzenie oraz korektę decyzji.

Fundamentalnym produktem 0!n jest zatem **zdolność przechodzenia od rozpoznanego przedmiotu do jawnego modelu, od modelu do uzasadnionej zmiany i od wyniku zmiany do nowego odczytu**. Artefakt jest jednostką, która pozwala utrzymać ciągłość tego obiegu. Każdy kolejny przyrost ma sens o tyle, o ile powiększa sprawdzalną zdolność systemu, zachowując wiedzę o tym, skąd ta zdolność pochodzi i gdzie kończy się jej zakres.

Źródła i podstawa weryfikacji

[1] Pakiet `lion-rag30-v1.5-r1`: `00_START_HERE.md`, `03_STATE_AND_CONTINUATION.md`, `29_PACKAGE_MANIFEST.json`. Kopie przekazane w `project_sources/`. Manifest: 29 plików treści oraz sam manifest, cutoff `2026-10-04T16:09:49.538029+00:00`. Kontrola zgodności wymienionych plików: bez rozbieżności.

[2] LION Polyhedral Studio R6, pakiet 0.6.0, archiwum `LION_POLYHEDRAL_STUDIO_R6.zip`. SHA-256: `3848fd50b9c2db9d60e0aee3a0204d905786f90ea9da71ed90bad194d4ec9b60`. Źródła lokalne: `development/LION_POLYHEDRAL_ARTIFACT_R1`. Wydanie kodu jest odrębne od historycznego Autonomous Implementation R6 oraz od wersji pakietu wiedzy 1.5.

[3] Pakiet wiedzy: `04_ARCHITECTURE_AS_IS.md`, sekcje „Model, który rzeczywiście wynika ze źródeł” i „Trzy perspektywy tego samego systemu”; `06_SEMANTIC_LANGUAGE.md`, sekcje „Jednostką współpracy jest identyfikowalne znaczenie” i „Istniejące operacje, nie nowy język wykonywania”.

[4] Studio R6: `src/lion_studio/model.py`, `normalize_workspace`, `validate_workspace`; `src/lion_studio/topology.py`; `docs/PHILOSOPHY_0N_PL.md`. Granice rodzica, abstrakcji i położenia należy odczytywać w odpowiadających im strukturach.

[5] Studio R6: `src/lion_studio/artifact_core/readers.py`, `ReaderRegistry`, `observe_value`, `Observation.from_dict`, `_document_reader`, `_catalog_reader`. Wbudowane czytniki i kwalifikacje odczytu.

[6] Studio R6: `src/lion_studio/model.py`, FACES; `src/lion_studio/artifact_core/profiles.py`, `ReadingProfile`; `docs/ARTIFACTIZATION_AND_EVOLUTION_R6_PL.md`, sekcje ontologii, geometrii i profilu odczytu.

[7] Studio R6: `src/lion_studio/artifact_core/profiles.py`, `SEMANTIC_AXES`; `docs/R4_SOURCE_ANALYSIS_PL.md`, „HMK-9D: rzeczywista definicja”; pakiet wiedzy `06_SEMANTIC_LANGUAGE.md`, „HMK-9D i holomozaika”.

[8] Studio R6: `src/lion_studio/artifact_core/projection.py`, `artifact_id`, `_reading`, `artifactize`, `project_from_readings`. Kwalifikacja projekcji, parametry, źródła i jawne relacje.

[9] Studio R6: `src/lion_studio/artifact_core/safety.py`, `canonical`, `pointer`, `type_matches`, `ReadingError`. Ograniczone wartości JSON i deterministyczne kodowanie UTF-8.

[10] Studio R6: `src/lion_studio/self_reading.py`, `read_self_model`, `_static_calls`, `_add_signature_ports`; `src/lion_studio/importers.py`, `_PythonReader._signature`; `configs/self-model.json`; `docs/SELF_MODEL_AND_MIGRATION_R6_PL.md`. Rozbieżność `returns` / `return_annotation` jest uwagą implementacyjną dotyczącą badanego wydania.

[11] Pakiet wiedzy: `11_FACTORY_BEANS_AND_MOSAIC.md`, wszystkie trzy sekcje merytoryczne; `20_ARCHITECTURE_REQUIREMENTS.md`, „Cel architektury rozwojowej” i „Macierz śledzenia”. Pojęcia fabryki opisują wymagania i kierunek rozwoju, z zaznaczonym zakresem istniejących wyników.

[12] Studio R6: `src/lion_studio/artifact_evolution.py`, `compile_process`, `evolve`; `src/lion_studio/model.py`, `evaluate_gate`, `simulate_workspace`; pakiet wiedzy `15_EVOLUTION_AND_HOMEOSTASIS.md`. Harmonogram projektowy i ograniczona pętla generacji.

[13] Studio R6: `src/lion_studio/artifact_service.py`, `ReadingStore`, `ArtifactService.dispatch`; `src/lion_studio/core_api.py`; `src/lion_studio/routing_broker.py`; `src/lion_studio/studio_broker.py`; `docs/CORE_API_AND_ROUTING_R4_PL.md`. Konkretne zakresy materializacji należy odczytywać z providera, planu i wyniku.

[14] Pakiet wiedzy: `12_AUTHORITY_ACTION_RUNTIME.md`, „Granice nie są jedną flagą READY” i „Publikacja nie jest deploymentem”; `13_PROVENANCE_MEMORY_AND_EVIDENCE.md`; `16_FAILURE_RECOVERY_AND_OBSERVABILITY.md`; `10_LPCL_AND_MISSION_CONTROL.md`. Zapis, wykonanie, obserwacja i ukończenie mają odrębne zakresy dowodowe.

Przykłady kodu są małymi przykładami dokumentacyjnymi dla przypiętego R6. Korzystają z istniejących funkcji i lokalnych nazw `example.*`; nie deklarują nowego standardu federacji. Praca nie zmienia kodu aplikacji ani stanu jej wdrożenia.